Interconnects

Tom Conte, Tushar Krishna

CS x290

Copyright © 2010-2014, 2017: Thomas M. Conte and Tushar Krishna

These notes MAY NOT BE REPRODUCED WITHOUT EXPRESS PERMISSION OF THE AUTHOR

Interconnect design dimensions

1. Topology
   - Physical architecture
2. Routing
   - Getting from here to there
3. Flow control
   - Dealing with congestion
4. Router architecture
   - Designing the switch

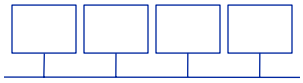
Topology: Direct vs. indirect networks

- Terminal node: source and sink of packets
- Switch node : forwards packets from input to output
- Direct network : single node type = terminal + switch
  - Although nodes usually are comprised of terminal interfaces to the switch, called the NI (network interface)
- Indirect network : two node types (terminal or switch)
  - Network is separate from terminals

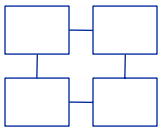
4

4

Topology: Shared vs. Point to Point



Shared, direct



Point to point, direct

- Can have direct & shared, indirect & shared, direct & point-to-point, indirect & point-to-point
- Note: broadcast/multicast possible in both systems
- But shared allows *observable* transactions
  - As stated before, for electrical property reasons, shared does not scale
- Large-scale network packets are broken into "flits": flow-control digits

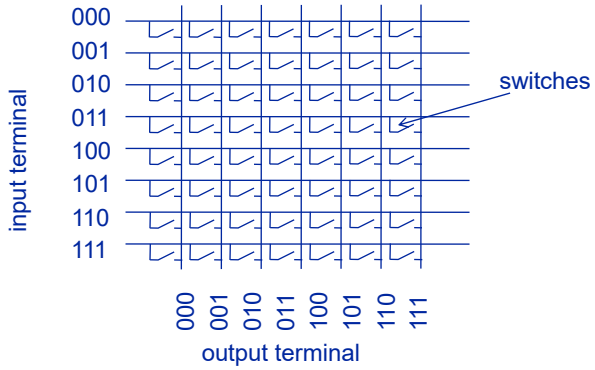
5

5

Definitions: Bisection

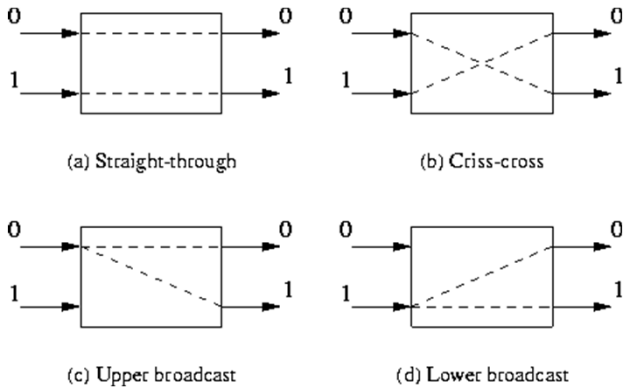
- A bisection of a network is a cut that divides the nodes into two equal (or equal +/-1 node) halves
- The bisection bandwidth is a measure of potential network performance
  - First find all bisections of a network
  - Bisectional b/w is the minimum (so that it is pessimistic) of the bandwidths across all bisections of a network

Example shared indirect network: crossbar



- Great cross-sectional bandwidth (ideal!)
- HUGELY expensive:
  - N^2 switches

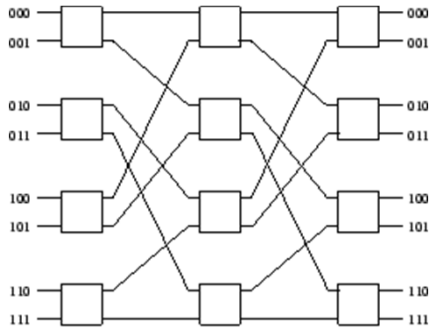
Example indirect network: shuffle-exchange network



8

8

Example indirect network: Omega network



9

9

Direct networks: Torus/Mesh

- K-ary *n*-torus or K-ary *n*-mesh
  - n dimensions
  - K nodes (switches) in each dimension (K-ary)
  - $N = K^n$ total nodes
  - Regular *n*-dimensional grid
  - A ring is n = 1
  - A binary n-mesh is k = 2 (aka a "*hypercube*")
  - Also called "K-ary *n*-cube torus or K-ary *n*-cube mesh" ("cube" is redundant)

10

10

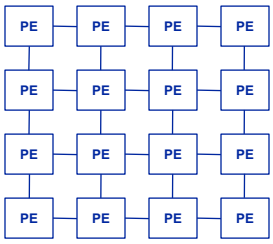
Torus vs. Mesh

- Each dimension of a torus has K nodes connected in a ring
- Each dimension of a mesh has K nodes connected in a line– last node is not connected to first node

11

11

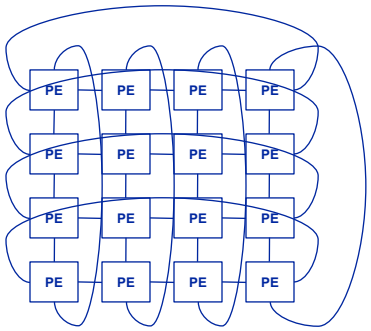
2-D mesh
(k = 4, n = 2) or 4-ary 2-mesh



12

12

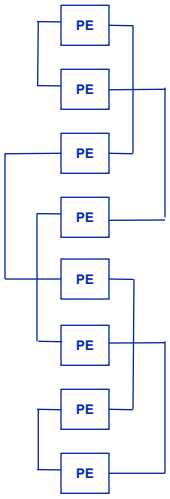
2-D torus
(k = 4, n = 2) also called a 4-ary 2-cube



13

13

Laying out a ring (or Torus, etc)



14

Routing

- Definition: ***Picking a route from source to destination***

Where Topology defines the performance *potential* (i.e., bisectional bandwidth),

Routing extracts as much performance as possible up to that potential

15

Routing algorithm super-classes

- Deterministic: always chooses same path from x to y even if multiple possible paths
- Oblivious: choose a route without considering network's present state

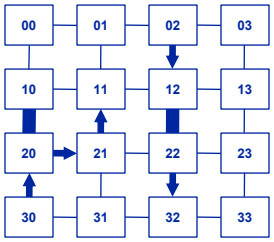
  NOTE: *all* deterministic algorithms are oblivious, *but not all* oblivious algorithms are deterministic
- Adaptive: adapt to state of the network, use the state information to make routing decisions
- From prior example:
  - Deterministic: Greedy (always go shortest path)
  - Oblivious: Greedy, Uniform and Weighted random
  - Adaptive: Load adaptive

17

17

Adaptive routing

- Network state of a channel typically measured as queue length at the destination
- Ex: Minimal adaptive routing: packets choose among set of shortest paths based on load

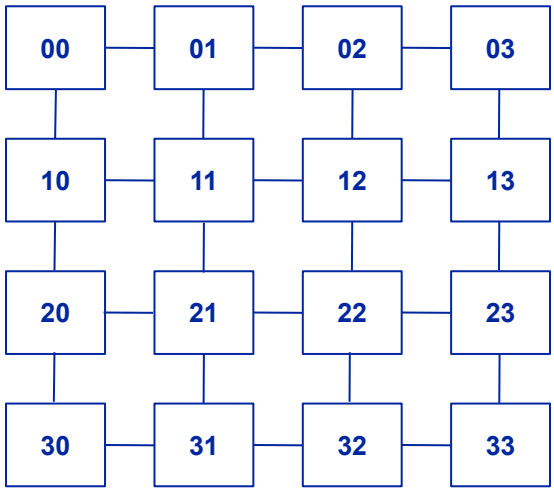


Route from 30 to 11 avoids congestion at 20-10

But route from 02 to 32 cannot avoid congestion at 12-22

18

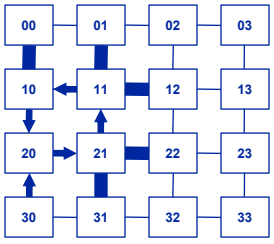
18



19

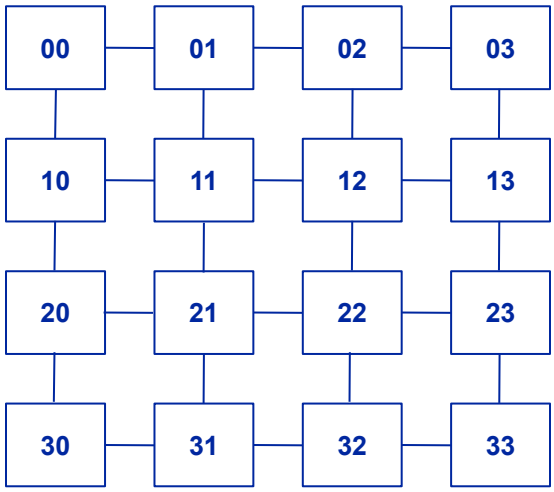
Fully adaptive routing

- Extend Minimal adaptive routing:
  - Allow non-shortest path routes
  - Can lead to "live lock" (here a route attempt from 30 to 02 that starts 30 to 20):



20

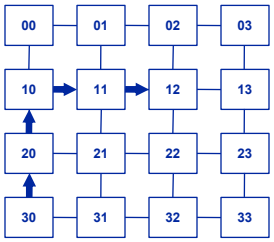
20



21

Deterministic: dimension-order routing

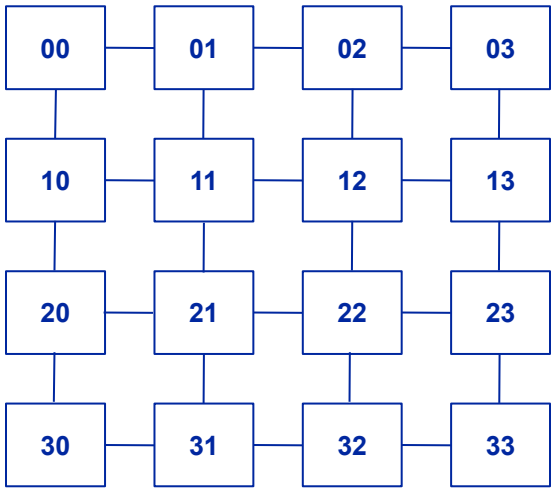
- For an k-ary n-torus or n-mesh, route each of n dimensions in turn
  - Greedily route shortest path in each direction
- Interpret the destination as a base k number and route in the direction of each digit of the destination



Route from 30 to 12 routes first in dimension 1 (3 to 1) and then in dimension 2 (0 to 2)

22

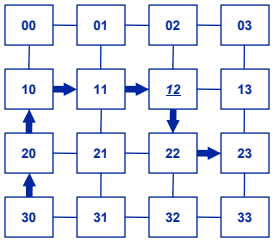
22



23

Oblivious routing: Valiant's algorithm

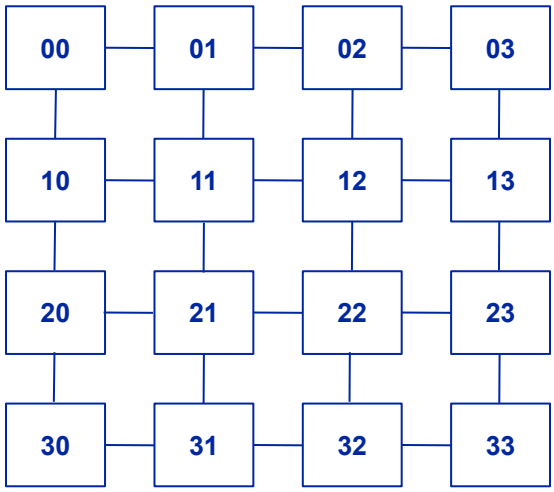
- Algorithm: to go from s to d, randomly pick a node, i, and first route deterministically from s to i and then from i to d
- Consider this where dimensional routing is used to get to and from the random node:



Route from 30 to 23 with random node 12

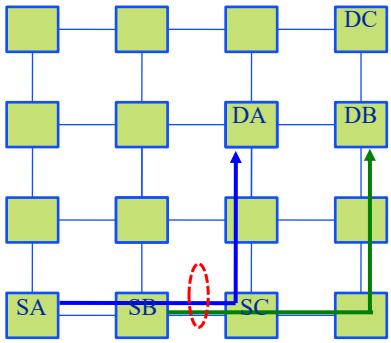
24

24



25

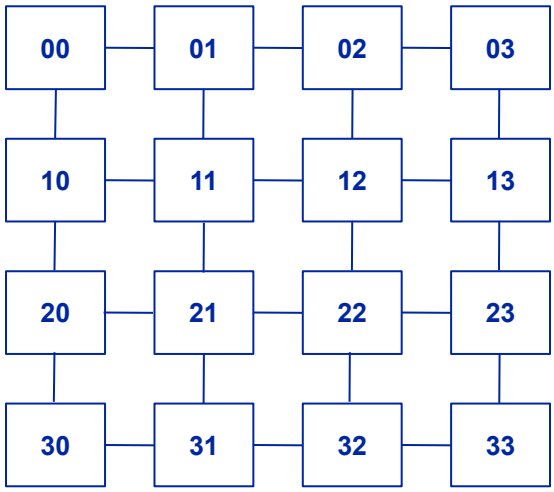
Circuit Switching



- Form a circuit from source to dest
- Probe to set up path through network
- Reserve all links
- Send data through links
  - Bufferless
- Tear down circuit once transmission complete

November 1, 2016

32



33

When does circuit switching make sense?

- When sending large chunks of data
- How large is the data being sent across the chip in shared memory NoCs?
  - Cache Line – 64B
  - Suppose
    - Link Width – 128b
      - 64x8/128 = 4 chunks of data
    - 1-cycle per hop
      - Setup Time: 3 cycles
      - ACK Time: 3 cycles
      - Data Transfer Time: 3 (for first chunk) + 3 (remaining chunks) cycles = 6 cycles
      - Total Time = 12 cycles
        - Half of the time went in circuit setup!

November 1, 2016

40

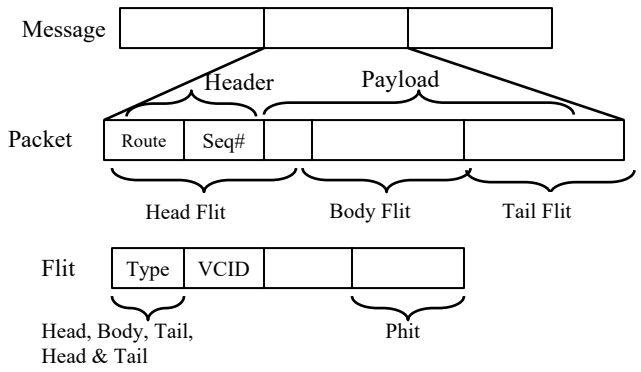
Packet Switching

- Messages composed of one or more packets
  - If message size is <= maximum packet size only one packet created
- Packets composed of one or more *flits*
  - Flit: flow control digit (i.e., there are x bits transferred across a link, then x = flit size)
- Flits composed of one or more *phits*
  - *Phit:* physical digit (i.e., 1 wire can hold 4 bits if multi-level or modulated)
- Link segments allocated at packet/flit/phit granularity

48

48

Packet

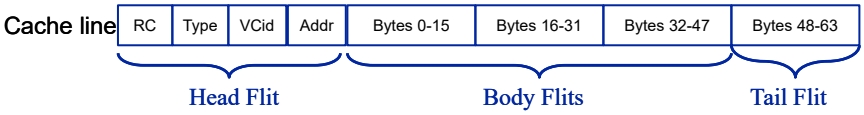


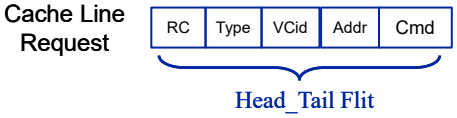
Off-chip: channel width limited by **pins** ➔ Requires phits
On-chip: **abundant** wiring means phit size == flit size

November 1, 2016

49

Typical NoC Packets





64B Cache Line
128-bit flits (i.e., link width)

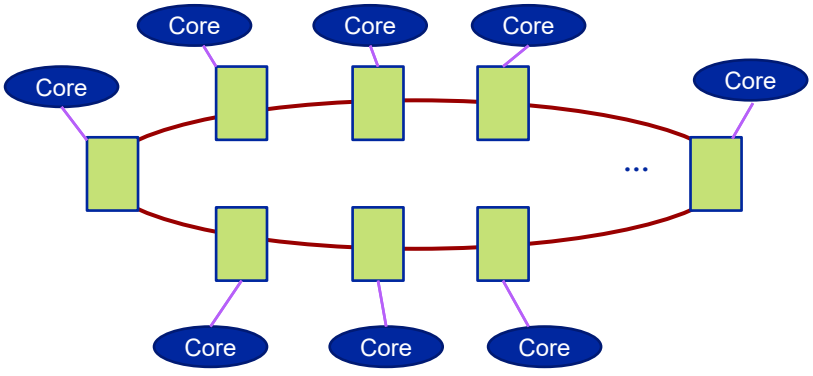
1 control flit (cache line req)
5 data flits (cache line data)

Packet contains destination/route information
Flits may not → all flits of a packet must take same route

50

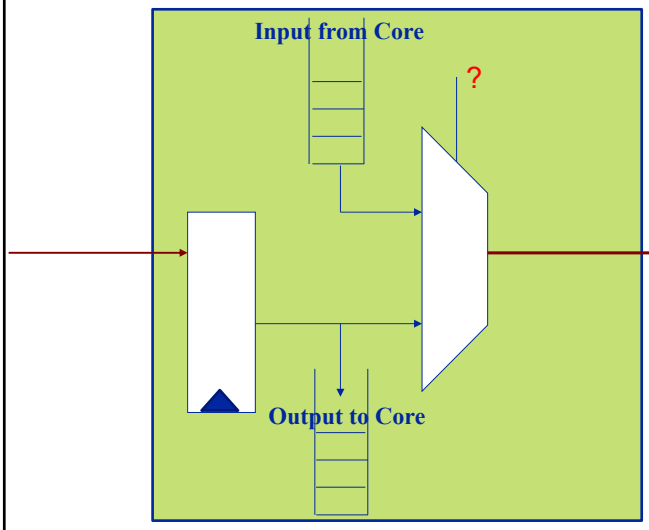
Designing the flow control

- Suppose we have a Ring network



51

Ring Stop



1. Who should use output link?

2. What to do with the other flit (from ring/core)

Have you seen this same situation in real life on a road network?

52

Flow Control: Link Arbitration



1. Who should use output link?

Traffic already on ring has priority

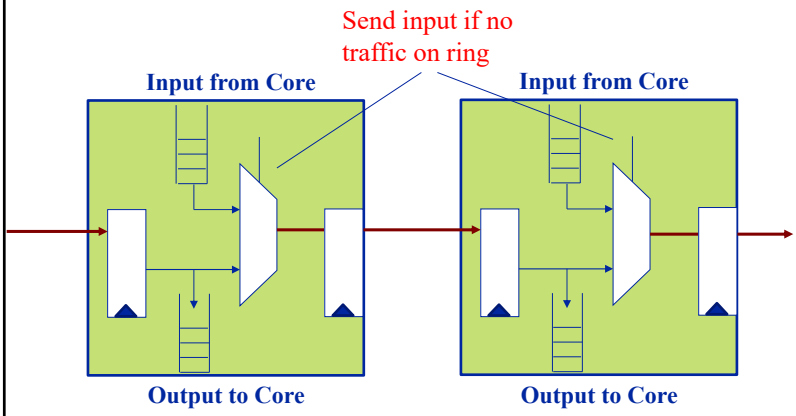
2. What to do with the other flit (from ring/core)

Wait

53

Flow Control Protocol

This is known as “arbitration”
The control structure is called an “arbiter”

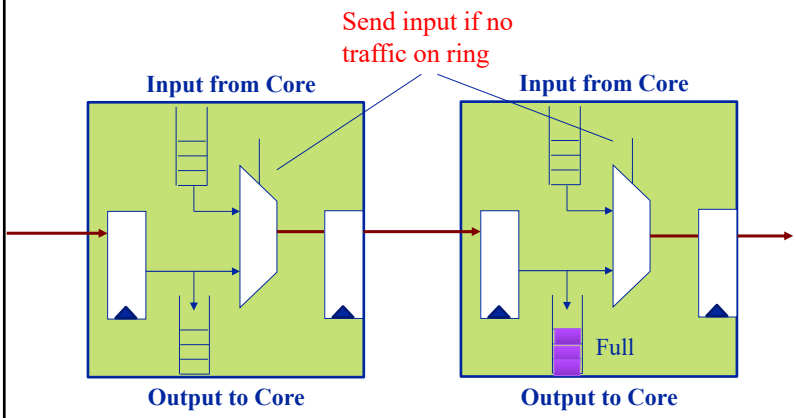


November 1, 2016

54

Flow Control Protocol

This is known as “arbitration”
The control structure is called an “arbiter”



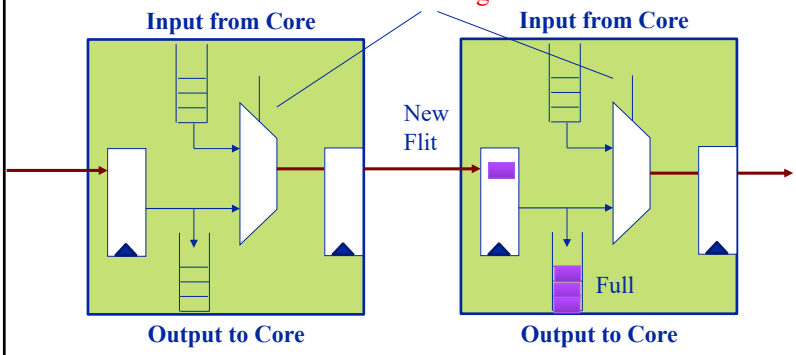
November 1, 2016

55

Flow Control Protocol

This is known as "arbitration"
The control structure is called an "arbiter"

3. What should a flit do if its output is blocked?



November 1, 2016

56

Flow Control: Buffer Management

- What should a flit do if its output is blocked?
  - Option 1: Drop!
    - Send a NACK back for dropped packet or have a timeout
      - Source retransmits
      - Implicit congestion control
    - Flow control protocol on the Internet
    - Advantage: can be bufferless!
    - Challenges?
      - Latency and energy overhead of re-transmitting more than that of buffering so not preferred on-chip

November 1, 2016

57

Flow Control: Buffer Management

- What should a flit do if its output is blocked?
  - Option 2: Misroute!
    - As long as N input ports and N output ports, can send flit out of some other output port
      - called "bouncing" on a ring
    - Advantage: can be bufferless!
    - Challenges
      - Energy
        - Routes become non-minimal – more energy consumption at router latches and on links
      - Performance
        - Non-minimal routes – can lead to longer delays
      - Correctness
        - Livelock! – cannot *guarantee* forward progress
          - Not the same as deadlock
          - *Need to restrict number of misroutes of same packet*

Flow Control: Buffer Management

- What should a flit do if its output is blocked?
  - Option 3: Wait!
    - Signal to previous router to not send any more flits till the input at this router can be drained
    - Backpressure techniques
      - On/Off : one bit to signal if next router can receive or not
        - Challenge: Delay of on/off signal
      - Credit-based : A count of how many flits can be sent to the next node
        - When should credit be decremented?

          Whenever a flit is sent to the next router
        - When should credit be incremented?

          Whenever a flit leaves the next router

Virtual Channels

- When a message blocks, instead of holding on to physical links so others can't use them, hold on to virtual links
- Multiple queues in buffer storage
  - Analogous to lanes on the highway
    - "Right-only" lane so that traffic going straight does not block
- Disadvantages?
  - More complex router
  - Arbitration between VCs required

Flow Control Protocols

- Bufferless
  - Circuit switching
  - Dropping
  - Misrouting
- Buffered
  - Store-and-forward
  - Virtual cut-through
  - Wormhole
  - Virtual-channel



Complexity & Efficiency

Summary: interconnects for manycores

- Three design decisions: Topology, Routing, Flow Control
- Manycores use point to point networks generally
- The network topologies, routing, flow control and deadlock avoidance all parallel work done in the 1990s

88